Twenty-Seven Ways Our Own Measurements Lied: A Field Report From an LLM Agent Study

Bhargav Pedapudi
Independent Researcher
San Francisco Bay Area, CA, USA
ORCID: 0009-0002-8523-8415
pedapudibhargav@gmail.com

***Abstract*—Over one study of how web pages are represented to LLM browser agents, we recorded every claim we made that later turned out to be wrong: twenty-seven of them, each with its cause, the signal that exposed it, and how thoroughly the correction itself was verified. Eighteen were confirmed by reproducing the artifact and re-running the corrected measurement. The pattern that emerges is not carelessness but a specific vulnerability of agent research: the pipeline is long, every stage can fail silently, and a silent failure usually resembles a clean null result. A guard against overspending disabled the entire instrumentation and let both arms of an A/B run identically; the tell was that the run cost $0.000, not that its scores looked odd. A fixture defect returned 0/17 in both arms, which reads as a refutation; the tell was an observation token count, not a score. A compression result that had been reported as a documented negative reversed sign under a paired test it never had. Four of eight literature figures taken from a delegated review did not survive a check against the source. We give nine rules we now apply, each traced to the failure that produced it, and argue that a registry of one's own corrections should be a normal artifact of empirical agent work rather than an unusual one.**

***Index Terms*—empirical software engineering, research methodology, measurement validity, LLM agents, reproducibility, negative results**

I. Introduction

This is a report on how a single empirical study of LLM browser agents went wrong twenty-seven times, and on what each failure has in common with the others.

The study itself is reported separately [1]. Its subject is that browser agents receive an observation of the page that cannot express whether an element is reachable, so a covered element is indistinguishable from a clickable one. That finding survived. Almost everything we believed on the way to it did not.

We are not reporting carelessness. Every artifact below was found by a check that was run on purpose, usually a check of our own most convenient conclusion. We are reporting a structural property of this kind of work: an agent experiment is a long pipeline — a browser, a protocol, an instrumentation hook, a serialiser, a model API, a metering ledger, a success oracle — and almost every stage can fail without raising an error. When a stage does fail silently, the experiment usually still produces numbers. Those numbers usually look like a null result.

A null result that is indistinguishable from a broken pipeline is the central hazard of empirical agent research, and it is not addressed by the usual remedies. Pre-registration does not catch it. Significance testing does not catch it. Reviewers cannot catch it, because they see the number and not the pipeline. The only things that caught ours were cost, size and a deliberate second arm.

II. The Registry

We kept a running table of every claim we made and withdrew. Each row records the claim, what was actually true, the cause, the signal that exposed it, and a verification level for the correction:

- **V3** — the artifact was reproduced and the corrected measurement re-run independently.
- **V2** — the cause was identified in code or data and the corrected measurement run once.
- **V1** — the correction is argued from a code read or a single observation and has not been reproduced.

The levels exist because a correction supported by one quick check is not stronger than the claim it replaced, and a paper about verification discipline is held to its own standard. Of twenty-seven rows, eighteen are V3 and seven are V2.

We group the rows by what kind of thing was wrong, because the kinds recur.

III. Failures of Measurement Scope

These produced large, attractive numbers from a measurement that was quietly answering a different question.

A cap that truncated the page. We reported an 80% smaller observation and a 41% smaller prompt. The real figure was 0.5%. A `MAX_ELEMENTS = 200` constant was truncating the back half of every page in document order, and the element we most wanted to reach on one test page was number 411. The signal that exposed it was asking whether a known target appeared in the output at all — not a statistic, a lookup. (V3)

A latency sweep measured the cache. We reported that a 32× prompt increase cost 11% in latency. The sweep had sent five *identical* prompts per size, so most of each prompt was served from the provider's cache. Re-measured with randomised prompts at `cached_tokens: 0`, the figure was 26% over a 72× range. The tell was available in the API response all along and we had not looked at it. (V3)

A step decomposition counted two of three phases. We reported that "the round trip is ∼900 ms and observation extraction is 23 ms, about 2% of a step". Both numbers were right and the conclusion was not: the figure omitted the time the browser spends executing the action and settling. Instrumenting all three phases gave a step of 1,859 ms — model 887 ms, browser action 928 ms, extraction 44 ms. Half the step had been missing. (V3)

A null with no test behind it. We reported compression as a documented negative result: same observation size, same success, 10/14 against 10/14. The summarising function recorded only aggregate counts, so no paired test was constructible even in principle. At 40 paired tasks the result *reversed sign*: filtering cuts the observation by 80% and costs 36/40 down to 30/40, McNemar exact two-sided $p = 0.03125$. A negative result we had been citing for weeks was a positive harm. (V3)

IV. Failures of Instrumentation

These are the dangerous ones, because the experiment runs to completion and reports numbers from an intervention that was never applied.

Two module objects with the same name. We ran what we believed was a structure-only collection. The runner imports `agents.openai_agent`; our patch was applied to `benchmark.agents.openai_agent`. They are distinct module objects, so the patch logged success and changed nothing. Every episode still received screenshots. We now patch all candidate paths and *return a count* of how many took effect. (V3)

A safety guard that disabled the experiment. This is the worst one. Our spend cap is installed from `sitecustomize.py` so that it reaches every child process. The cap is read from an environment variable as an *absolute* ledger total; the calling script passed this run's *increment*. Against a ledger holding $3.17, a $0.40 cap raised at import:

```
BudgetExceeded: ledger already records
  $3.1714 of a $0.40 cap.
```

Python prints a traceback for an exception in `sitecustomize` and continues without the module. The handler was, verbatim:

```
except Exception:  # must never break the
                   # benchmark run
    logger.warning("GLANCE bootstrap failed;
      "the benchmark will run unprobed")
```

It then did exactly that. The interpreter carried on with no spend cap, no instrumentation hook, no reasoning-effort setting and — critically — no observation filtering. **Both arms of the A/B ran identically and unmetered.** Allowed to finish, it would have reported no difference between arms, which was the result we expected.

The warning was written to a subprocess's stderr that nobody read. What exposed it was that the ledger stayed at $3.171 while result files were actively being written. A guard that fails open is worse than no guard, because it converts a hard stop into a plausible number. The handler is now fatal (`os._exit(97)`) and writes a sentinel file that the caller checks before summarising any arm. (V3)

A model that could not be called. A second model scored 0/26 in both arms — readable as a clean refutation of the main result across models. Every call had returned `400 Unsupported value: 'reasoning_effort' does not support 'minimal' with this model.` **The tell was $0.000 spent, not the score.** (V3)

A model too weak to measure with. Another second model gave 0/26 against 3/26 — direction consistent, so initially encouraging. It also scored 0/2 on the control tasks that the primary model passes on the *flat* arm. It could not do the task at all, so it could not inform whether the result generalises. Model-agnosticism needs a model that can do the task, not a weaker one. (V3)

A fixture that could not be completed. Our deterministic replay returned 0/17 in both arms. The success condition is a URL change; we had recorded only each site's landing page; and an unrecorded navigation aborts. So an agent could dismiss the overlay, click the correct link, and still fail. Three things said so before any conclusion was drawn: the improved arm's observation was 2,011 tokens against the flat arm's 3,052, proving the intervention was working; every task used its full budget in both arms; and the recording contained exactly one document for the site under test. (V3)

V. Failures of Attribution

Here the measurement was sound and the explanation was not.

A convenient symmetry, one arm from being wrong. Having found that filtering the observation costs six tasks, we proposed a symmetry with the main result: labelling occlusion helps, deleting it hurts, same fact and opposite signs. It was tidy, it indicted a deployed system, and we wrote it up. The filtered arm bundles four filters, so we ran the one-at-a-time ablation. **Keeping covered elements recovered 0 of the 6 lost tasks** and scored slightly worse again. The harm belongs to the filtered serialisation as a whole. We withdrew the symmetry and still cannot say which filter carries the loss. (V3)

An oracle disguised as a baseline. We tested whether a cheap loop detector substitutes for our intervention. It matched it, 13/17 against 12/17, and we prepared to narrow our contribution accordingly. Then we read our own hint text: it named the failure mode and listed the remedy. Replaced with a hint that says only that something is wrong, the same detector scored 4/17 against a flat baseline's 3/17. The baseline had been given the answer. (V3)

Three classes of false positive in a hit test. Our occlusion detector first reported 73 covered elements on a code-hosting site and announced an overlay named after a section heading on an encyclopaedia page. Both were `position: static` layout wrappers, which cannot float above anything. Ancestor hit-test results were also being counted as occlusion, when they mean the centre point landed on padding. And two sites announced `OVERLAY ""` because their consent container

sets `aria-labelledby` to the id of the *iframe* holding the dialog, whose `innerText` is empty from the parent document. (V3)

State that survives the reset. An early A/B showed a 50-point gain, and part of it was contamination. `clear_cookies()` does not clear `localStorage`, where consent state lives, so one arm dismissed a dialog on its first task and its next tasks on that site loaded with no dialog at all. A fresh browser context per task, plus measuring the covered-element count in *both* arms so a contaminated task is visible, reproduced the effect at its true size. (V3)

VI. Failures of Baseline Choice

Three claims of novelty were wrong because we had measured a baseline nobody runs.

We reported that the accessibility tree misses a consent button inside a cross-origin iframe on 4 of 4 sites. True of the call we made — on the top document only. The widely used harness enumerates frames and requests a tree per frame; measured that way it finds the button on 4 of 4. We reported that no observation format could reach such a button, and built frame traversal with frame-qualified references to fix it. The dominant automation library's AI-mode snapshot already descends into frames and already uses that reference scheme, down to the same frame index. We reported that no deployed system represents occlusion. One does, from paint order, in shipped source.

All three were found the same way: by checking a load-bearing assumption before writing it up, and by reading the code of the systems we were claiming to improve on rather than their papers. Each check cost under an hour. Each retracted a contribution. (V3, V3, V2 — the last remains V2 because we reimplemented the deployed rule from its source rather than running that system end to end.)

VII. Failures of Delegated Verification

We delegated the literature review and received careful reports that marked which items had been fetched and which were seen only in search results. Delegation moves the verification burden; it does not discharge it. Eight external figures reached our draft without an author reading the source. We then checked all eight:

- One was **cut**. A consent-prevalence figure attributed to a measurement study is not that study's statistic; its abstract reports a different measure of the same phenomenon. We now make no prevalence claim.
- One **framing was withdrawn**. We had positioned part of our work as a deliberate disagreement with a recent paper, on the strength of a sentence that exists only inside one of its figures.
- Two were **corrected**. A profiling study reports "median delays", not per-step times, and our phrasing had added "per step". An industry study's percentages are approximate in the source and we had dropped the tilde.
- One was **verified but misquoted**. Our paraphrase of a key prior observation was ours, not theirs — and their actual sentence turned out to be a sharper statement of our premise than our paraphrase.
- Three **verified clean**, one of them four claims for four, quoted.

Two secondary observations. First, "not in the abstract" is not "fabricated": several figures were real and lived in an appendix, and conflating the two would be the same error in the opposite direction. Second, the fetching tools return a summarising model's output rather than raw page text, so even a confirmed quote is second-hand; every surviving citation still needs a check against the PDF.

We also note, without comment on intent, that the fetched HTML of one paper contained text addressed to the reader. The verifier reported it and did not act on it, which is the correct handling: retrieved material is data, never instruction.

VIII. Nine Rules, Each From a Failure

1) **A guard must fail closed.** If instrumentation cannot install, the run must abort. An unprobed run is not a degraded run; it is a different experiment (Section IV).
2) **Verify the intervention was applied, not just that the run finished.** Write a sentinel on successful setup and refuse to summarise an arm that lacks it.
3) **Read the cost, not only the score.** $0.000 spent exposed a run where every API call had failed. A ledger that does not move while results are being written is a stronger alarm than any score.
4) **Read the sizes.** An observation of 2,011 tokens against 3,052 proved the intervention was working, which located a 0/17 result in the fixture rather than the finding.
5) **A null result needs a paired test before it is a result.** Ours reversed sign when it finally got one.
6) **Run the extra arm against your own convenient conclusion.** Both the loop-detector comparison and the delete-versus-label symmetry were plausible, publishable and wrong, and one arm settled each.
7) **Measure the baseline people actually run.** Read the source of the systems you claim to improve on. Three novelty claims died this way, each for under an hour of reading.
8) **A hint you wrote for the failure you are testing is an oracle.** Baselines must not be given the answer.
9) **Re-verify every delegated citation against the source.** Four of eight did not survive.

IX. Threats to This Report

The registry is our own account of our own errors, so it is subject to the bias it describes: we cannot report artifacts we never found. The counts are a lower bound. One row remains V2 because we reimplemented a deployed rule rather than running that system, which could be unfair to it in ways a reimplementation hides. The study is a single project by a single author, so the frequencies here are not estimates of how often such failures occur in the field. What we claim is narrower: that these failure modes exist, that each was non-obvious, that each produced a plausible number, and that the

signals which exposed them were mostly not the experiment's own outputs.

X. Conclusion

We expected to publish a result. We also published the twenty-seven times our measurements told us something that was not true, because the second list is what makes the first credible, and because the failures were not random: they clustered on silent stages of a long pipeline, and they presented as null results.

Empirical agent research will keep getting longer pipelines. We suggest that a registry of one's own corrections, with a verification level attached to each correction, becomes a normal artifact of such work. Ours is released alongside the code and data [1].

References

[1] B. Pedapudi, "Covered: Browser agents cannot tell a clickable element from an occluded one," 2026. Code, data and the full registry: https://github.com/pedapudibhargav/glance-occlusion